

Fullstack Travel App Project Plan

Fullstack Travel App - Graduation Project Plan

1. Project Overview

A fullstack travel application built with **React**, **JavaScript**, **Node.js/Express**, and two external APIs:

- **Wikivoyage via Wikimedia Enterprise API** - provides travel guides, tips, and city information.
- **Geoapify Places API** - provides maps, POIs (points of interest), geocoding, and autocomplete.

The app will allow users to:

- Search for cities
 - View interactive maps
 - Explore attractions and POIs
 - Read travel tips from Wikivoyage
 - Navigate through city-specific pages
-

2. High-Level Architecture

Frontend (React)

- React components for map, search, city pages
- MapLibre GL for interactive maps
- Geoapify Autocomplete for city search
- Axios for API calls
- React Router for navigation

Backend (Node.js + Express)

- Proxy server for Wikimedia Enterprise API
- Optional: user authentication, favorites, itineraries

Data Sources

- Geoapify Places API
- Wikimedia Enterprise API (Wikivoyage dataset)

3. Development Environment Setup

Required Tools

- Node.js (LTS)
- VS Code
- Git + GitHub

Create React App

```
npx create-react-app travel-app  
cd travel-app  
npm start
```

Install Dependencies

```
npm install maplibre-gl  
npm install @geoapify/geocoder-autocomplete @geoapify/react-geocoder-autocomplete  
npm install react-router-dom  
npm install axios
```

4. API Keys Setup

Geoapify

- Create account at geoapify.com
- Generate API key

Wikimedia Enterprise API

- Request access at enterprise.wikimedia.com

Environment Variables

Create .env in React:

```
REACT_APP_GEOAPIFY_KEY=your_key_here  
REACT_APP_BACKEND_URL=http://localhost:4000
```

Create .env in backend:

```
WIKI_API_KEY=your_wikimedia_key
```

5. Backend Setup (Node.js + Express)

Install Backend Dependencies

```
mkdir backend
cd backend
npm init -y
npm install express axios cors dotenv
```

Basic Server

```
const express = require("express");
const axios = require("axios");
require("dotenv").config();
const app = express();
app.use(require("cors")());

app.get("/api/wikivoyage/:city", async (req, res) => {
  const city = req.params.city;

  try {
    const response = await axios.get(
      `https://api.enterprise.wikimedia.com/v2/wikivoyage/page/${city}`,
      {
        headers: { Authorization: `Bearer ${process.env.WIKI_API_KEY}` }
      }
    );

    res.json(response.data);
  } catch (err) {
    res.status(500).json({ error: "Failed to fetch Wikivoyage data" });
  }
});

app.listen(4000, () => console.log("Backend running on port 4000"));
```

6. Interactive Map Setup

Map Component

```
import { useEffect, useRef } from "react";
import maplibregl from "maplibre-gl";

export default function Map() {
  const mapContainer = useRef(null);

  useEffect(() => {
    const map = new maplibregl.Map({
      container: mapContainer.current,
      style: `https://maps.geoapify.com/v1/styles/osm-carto/style.json?apiKey=${process.env.REACT_APP_GEOAPIFY_KEY}`,
      center: [-0.1276, 51.5074],
      zoom: 12
    });
  }, []);

  return <div ref={mapContainer} style={{ height: "100vh" }} />;
}
```

7. Geoapify Places API Integration

Example Request

```
https://api.geoapify.com/v2/places?categories=tourism.attraction&filter=circle:lon,lat,radius&limit=50&apiKey=
```

Use this to:

- Fetch POIs
- Add markers to the map
- Display popups with name and category

8. Geoapify Autocomplete Search

Autocomplete Component

```
<GeoapifyContext apiKey={process.env.REACT_APP_GEOAPIFY_KEY}>
  <GeoapifyGeocoderAutocomplete
    placeholder="Search for a city"
    placeSelect={(place) => console.log(place)}
  />
</GeoapifyContext>
```

When a user selects a city:

- Center the map
 - Fetch Wikivoyage article
 - Fetch Geoapify POIs
-

9. City Page Structure

React Router

```
<Route path="/city/:name" element={<CityPage />} />
```

City Page Features

- Fetch Wikivoyage article
 - Display sections:
 - Overview
 - Get in
 - See
 - Do
 - Eat
 - Sleep
 - Interactive map with POIs
-

10. Optional Features

- Favorites system
- User login (JWT or Firebase)
- Itinerary generator

- Dark mode
 - Offline caching
-

11. What You Need to Provide Next

To continue building the app:

1. Choose your MVP features
2. Decide if you want a backend
3. Decide if you want folder structure generated
4. Decide if you want coding to begin

This document contains everything needed to begin development and can be saved as your project plan.